



NovaMarkets Vault

Security Review



Disclaimer

Security Review

NovaMarkets Vault



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

NovaMarkets Vault

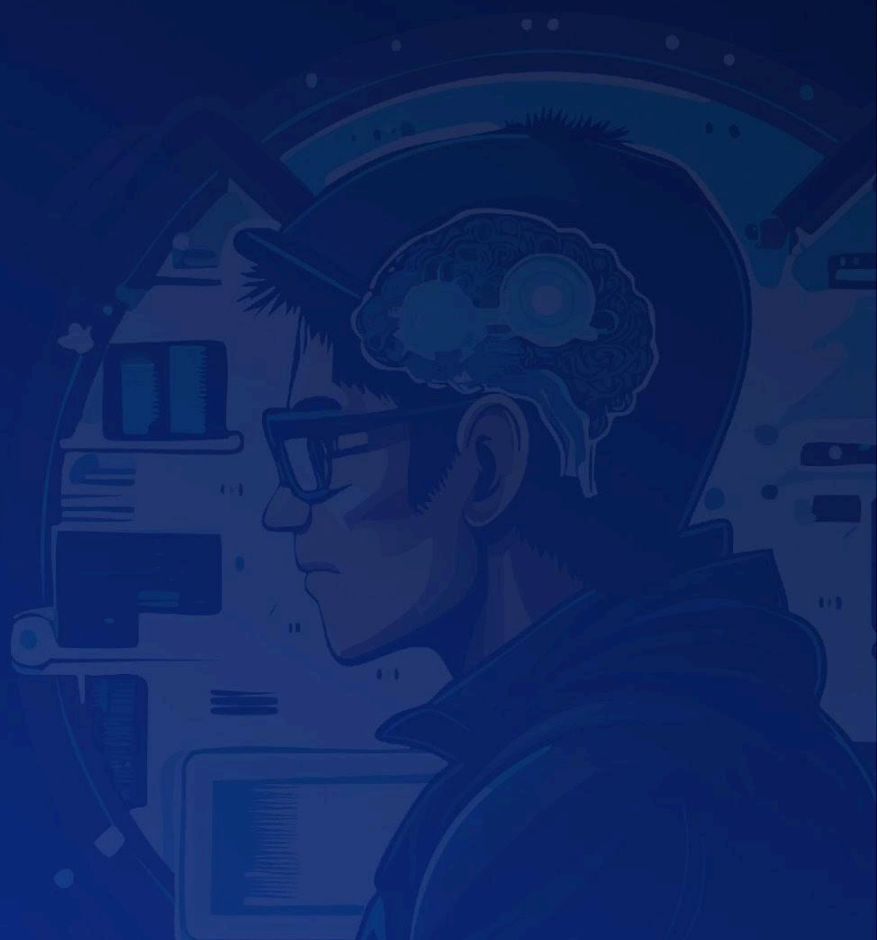


Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S-C01	18
3S-H01	20
3S-H02	22
3S-H03	24
3S-H04	26
3S-M01	28
3S-L01	30
3S-L02	32
3S-L03	34
3S-L04	36
3S-N01	38
3S-N02	40
3S-N03	42

Summary Security Review

NovaMarkets Vault



Summary

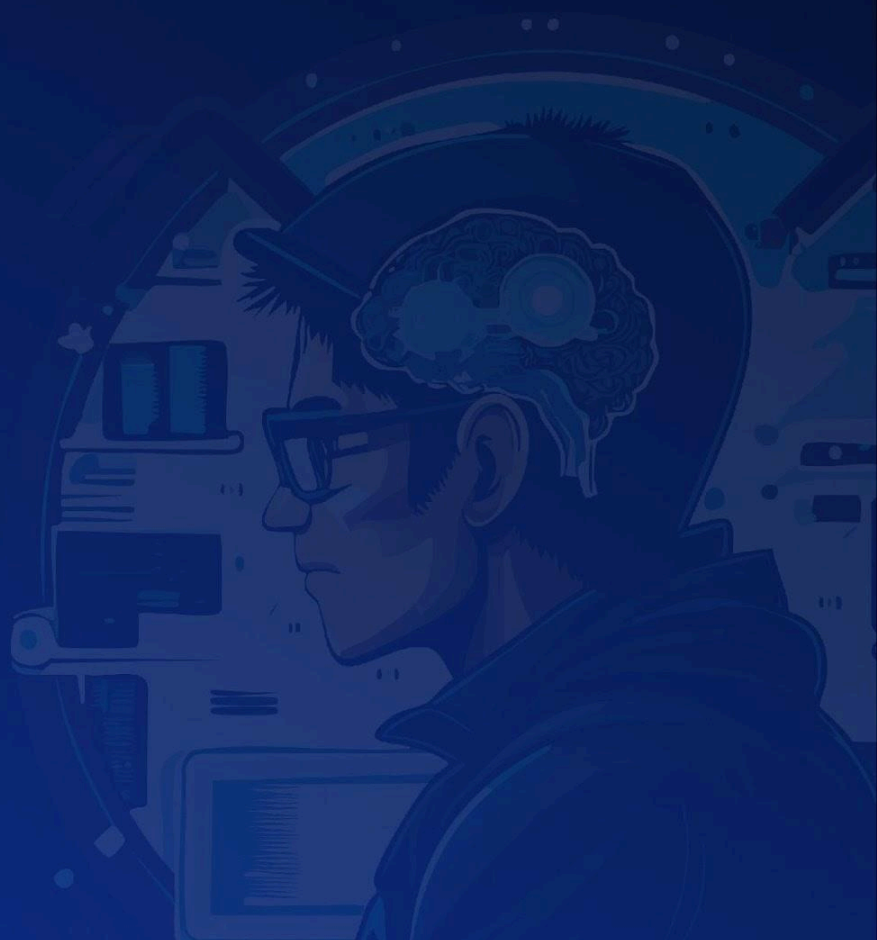
Three Sigma audited Nova Markets in a 0.8 person week engagement. The audit was conducted from 10/04/2026 to 11/04/2026.

Protocol Description

Nova Markets is a delta-neutral vault system built on HyperEVM that gives users a single-click entry into a market-neutral yield strategy executed on Hyperliquid. Users deposit whitelisted ERC-20 tokens and receive a share token whose value tracks an exchange rate maintained on-chain by a dedicated pricing and rate-update pipeline, with prices sourced from Hyperliquid's spot precompiles and stablecoin reference oracles. Deposited capital is pooled, bridged to Hyperliquid Core, and deployed by an off-chain solver into hedged spot-and-perpetual positions, while redemptions are handled asynchronously through a withdrawal queue that settles requests at the prevailing exchange rate.

Scope Security Review

NovaMarkets Vault

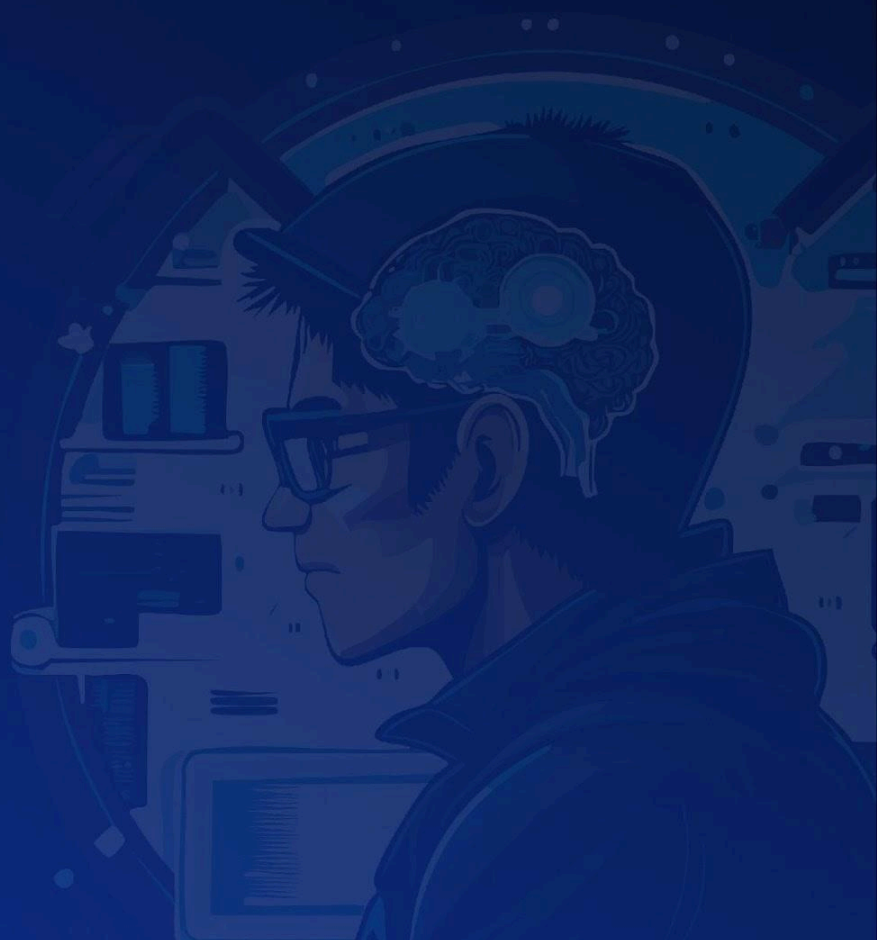


Scope

Filepath	nSLOC
src/vault/ExchangeRateUpdater.sol	354
src/vault/DnCoreWriterVault.sol	256
src/vault/Depositor.sol	105
src/vault/WhitelistedDepositor.sol	94
src/vault/WhitelistedVaultToken.sol	49
src/vault/SpotPxOracle.sol	19
src/vault/OneToOneOracle.sol	9
Total	886

Methodology Security Review

NovaMarkets Vault



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

NovaMarkets Vault



Project Dashboard

Application Summary

Name	NovaMarkets
Repository	https://github.com/Nova-Markets/DNCoreWriter
Commit	1887333
Language	Solidity
Platform	HyperEVM

Engagement Summary

Timeline	10/04/2026 to 11/04/2026
Nº of Auditors	2
Review Time	0.8 person weeks

Vulnerability Summary

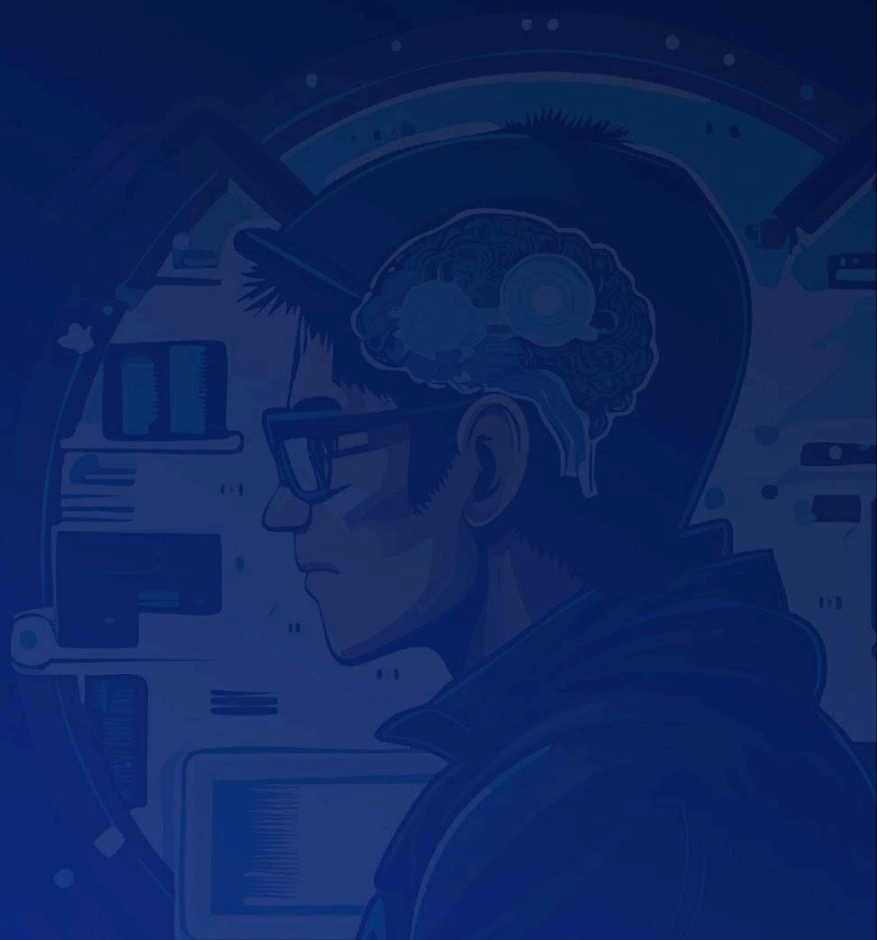
Issue Classification	Found	Addressed	Acknowledged
Critical	1	1	0
High	4	4	0
Medium	1	1	0
Low	4	4	0
None	3	3	0

Category Breakdown

Suggestion	3
Documentation	0
Bug	10
Optimization	0
Good Code Practices	0

Risk Section Security Review

NovaMarkets Vault



Risk Section

No risks were identified.

Findings

Security Review

NovaMarkets Vault



Findings

3S-C01

SpotPxOracle returns price in the wrong direction, causing heavy NAV miscalculation for non-USDC vaults

Id	3S--C01
Classification	Critical
Impact	Critical
Likelihood	High
Category	Bug
Status	Addressed in #836f7c0 .

Description

ExchangeRateUpdater::_calculateTotalAssets computes the vault's total net asset value by converting USDC-denominated spot position values and account margin into the vault's **baseAsset**. It does so by multiplying USDC amounts by **usdcToBaseAssetAggregator.getPrice()**, following the same convention used ten lines above for per-token **oracleAggregators**: the oracle returns a price in "baseAsset per input token" direction, and the caller multiplies.

This convention is correct and consistent across the codebase. The **OracleAggregator** contract computes **baseFeed.latestAnswer() / quoteFeed.latestAnswer()**. When configured for USDC-to-baseAsset conversion (e.g., **baseFeed = USDC/USD**, **quoteFeed = USDH/USD**), it yields baseAsset per USDC, which is the direction **_calculateTotalAssets** expects. **OneToOneOracle** returns **1e8**, which is trivially correct in both directions.

The problem is in **DeployWhitelistedVault.s.sol**. For non-USDC vaults, the deploy script creates a **SpotPxOracle** using the **baseAsset/USDC** Hyperliquid spot pair (line 164 comment confirms this) and passes it as **usdcToBaseAssetAggregator** (line 180). However, **SpotPxOracle::getPrice** returns **l1Read.spotPx(spotPxId)** directly, which for a USDH/USDC pair returns USDC per USDH (the baseAsset-to-USDC direction). The code needs the inverse: USDH per USDC (the USDC-to-baseAsset direction). The **SpotPxOracle** NatSpec at line 8 confirms this: "to price USDH in USDC terms, use the spotPxId for the USDH/USDC pair (230)".

Concretely, if 1 USDH = 0.50 USDC, **getPrice()** returns 0.50 when **_calculateTotalAssets** needs 2.0 (the reciprocal). A vault holding 1,000,000 USDC of spot positions computes 500,000 USDH instead of the correct 2,000,000 USDH. The error factor is the square of the price deviation from 1.0, and it flips direction: when baseAsset is cheap relative to USDC, NAV is deflated (enabling deposit arbitrage); when expensive, NAV is inflated (enabling withdrawal arbitrage). The bug is invisible at 1:1 parity, which is why **OneToOneOracle** USDC vaults are unaffected.

Recommendation

The fix belongs in **SpotPxOracle**, not in **_calculateTotalAssets**. Since Hyperliquid spot pairs are natively TOKEN/USDC (no USDC/TOKEN inverse pairs exist), **SpotPxOracle::getPrice** should invert the raw price so its output matches the USDC-to-baseAsset direction that the variable name and the codebase convention require.

```
function getPrice() external view returns (uint256) {
-   return uint256(l1Read.spotPx(spotPxId));
+   uint256 rawPx = uint256(l1Read.spotPx(spotPxId));
+   require(rawPx > 0, "SpotPxOracle: zero price");
+   return 10 ** (2 * uint256(pxDecimals)) / rawPx;
}
```

The **require** guard also addresses the zero-price risk. Note that the **pxDecimals** constructor argument must match **8 - baseSzDecimals** for the given Hyperliquid spot pair; for USDH/USDC (pair 230, **szDecimals=2**), the current value of 6 is correct.

3S-H01

Unchecked fee deduction underflow in **updateExchangeRate** leads to permanent denial of service

Id	3S--H01
Classification	High
Impact	Critical
Likelihood	Low
Category	Bug
Status	Addressed in #83094ae .

Description

ExchangeRateUpdater::updateExchangeRate is a keeper-permissioned function responsible for computing the vault's net asset value and propagating it to the **Pricer**. On each call it fetches the vault's **totalAssets** and subtracts **feesAccumulatedInBaseAsset** (the protocol fees accrued since the last claim) before converting the result to a per-share exchange rate.

The subtraction at line 189 is performed without an overflow guard:

```
totalAssets -= feesAccumulatedInBaseAsset;
```

Because **feesAccumulatedInBaseAsset** is monotonically non-decreasing between fee claims, and **totalAssets** can shrink independently (user withdrawals, losses on underlying positions, mass migration of depositors), there is a reachable state where **feesAccumulatedInBaseAsset > totalAssets**. When that condition holds, Solidity 0.8's default checked arithmetic causes an underflow revert. Since **updateExchangeRate** is the only path for refreshing the exchange rate, a revert here freezes the NAV and blocks all deposit and withdrawal operations that depend on a current rate. The read-only counterpart **ExchangeRateUpdater::updateExchangeRateView** at line 204 contains the identical unguarded subtraction.

The scenario does not require an exploit or external manipulation. A vault that has been operational long enough to accumulate significant fees, followed by a period of heavy

withdrawals or underlying asset depreciation, naturally reaches this state. The protocol's own **CreditFundExchangeUpdater** already handles this case correctly with a conditional check (lines 74-78), confirming the pattern was recognized elsewhere but missed in the vault variant.

Recommendation

Cap the fee deduction so that **totalAssets** floors at zero instead of reverting. This aligns **ExchangeRateUpdater** with the safe pattern already present in **CreditFundExchangeUpdater**.

```
- totalAssets -= feesAccumulatedInBaseAsset;
+ totalAssets = totalAssets > feesAccumulatedInBaseAsset
+   ? totalAssets - feesAccumulatedInBaseAsset
+   : 0;
```

Apply the same fix to **ExchangeRateUpdater::updateExchangeRateView** at line 204.

3S-H02

Missing **receive** function in **WithdrawalQueue** leads to permanent inability to release native HYPE for withdrawals

Id	3S--H02
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #986cd21 .

Description

DnCoreWriterVault::releaseTokensForWithdrawals is the privileged function (restricted to **WITHDRAWAL_ROLE**) responsible for transferring tokens from the vault to the **WithdrawalQueue** contract so that pending withdrawal requests can be fulfilled. When the token being released is the native asset HYPE (identified by the sentinel address **0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEE**), the function performs a native transfer via Solidity's **transfer** method:

```
if (token == HYPE) {
    payable(withdrawalQueue).transfer(amount);
}
```

However, the **WithdrawalQueue** contract does not declare a **receive()** or **fallback()** function. In Solidity, a contract without either of these functions cannot accept plain native transfers. As a result, **payable(withdrawalQueue).transfer(amount)** will always revert.

Recommendation

Add a **receive()** function to the **WithdrawalQueue** contract so it can accept native transfers:

```
contract WithdrawalQueue is Auth {
+   receive() external payable {}
+ }
```

using SafeTransferLib for address;

3S-H03

Stale **withdrawalQueue** in **ExchangeRateUpdater** causes incorrect idle balance calculation

Id	3S--H03
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #c24fa8a .

Description

ExchangeRateUpdater tracks idle token balances across the protocol, including tokens held by the withdrawal queue, to compute accurate exchange rates for the vault. The **withdrawalQueue** address is fetched once during construction at line 110 by reading **DnCoreWriterVault::withdrawalQueue()** and stored as an immutable-in-practice state variable.

However, **DnCoreWriterVault** exposes **DnCoreWriterVault::setWithdrawalQueue**, an admin-only setter that allows updating the **withdrawalQueue** address at any time. When the vault's withdrawal queue is updated, the **ExchangeRateUpdater** contract retains the old address because it has no corresponding setter or mechanism to refresh its cached copy. This causes **ExchangeRateUpdater::getIdleTokenBalances** (line 176) to query token balances of the stale address rather than the current withdrawal queue, producing incorrect idle balance figures and, consequently, incorrect exchange rate calculations.

Since the **ExchangeRateUpdater** contract provides no setter for **withdrawalQueue** and the value is only assigned in the constructor, the only remediation path after a withdrawal queue change on the vault would be redeploying the entire **ExchangeRateUpdater** contract.

Recommendation

Rather than caching **withdrawalQueue** as a local state variable, **ExchangeRateUpdater** should read it dynamically from the **DnCoreWriterVault** each time it is needed. Given that the protocol operates on Hyperliquid where gas costs are minimal, the additional external call is negligible. This ensures **ExchangeRateUpdater** always references the current withdrawal queue without requiring redeployment or manual synchronization.


```
- balances[i] +=  
IERC20Metadata(assetConfigs[i].token).balanceOf(withdrawalQueue);  
+ balances[i] += IERC20Metadata(assetConfigs[i].token).balanceOf(  
+   DnCoreWriterVault(payable(dnCoreWriter)).withdrawalQueue()  
+ );
```

The **withdrawalQueue** state variable and its constructor assignment can then be removed from **ExchangeRateUpdater**.

3S-H04

Missing **ADMIN_ROLE** grant to **finalAdmin** in deploy script causes permanent governance lock

Id	3S--H04
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #5180b63 .

Description

DnCoreWriterVault uses OpenZeppelin's **AccessControl** with a custom role hierarchy configured in **DnCoreWriterVault::initialize**. The **ADMIN_ROLE** is set as its own admin and as the admin of **BRIDGE_ROLE**, **WITHDRAWAL_ROLE**, and **TRADING_AGENT_ROLE**. Only holders of **ADMIN_ROLE** can call privileged functions such as **setCoreDepositWallet**, **setWithdrawalQueue**, **registerToken**, **toggleWithdrawalToken**, **toggleTradeableAsset**, **addAPIWallet**, **removeAPIWallet**, and **sendAction**. Only **ADMIN_ROLE** holders can grant or revoke any role, including **ADMIN_ROLE** itself.

During initialization, **ADMIN_ROLE** is granted to **msg.sender** (the deployer). The deploy script's **DeployWhitelistedVault::_finalizeSetupAndTransferRoles** then grants **BRIDGE_ROLE**, **WITHDRAWAL_ROLE**, and **TRADING_AGENT_ROLE** to **finalAdmin**, but never grants **ADMIN_ROLE**. The deployer retains it, and **finalAdmin** never receives it.

```
// DeployWhitelistedVault.s.sol:427-429
deployed.dnCoreWriter.grantRole(deployed.dnCoreWriter.BRIDGE_ROLE(),
finalAdmin);
deployed.dnCoreWriter.grantRole(deployed.dnCoreWriter.WITHDRAWAL_ROLE(),
finalAdmin);
deployed.dnCoreWriter.grantRole(deployed.dnCoreWriter.TRADING_AGENT_ROLE(),
finalAdmin);
// ADMIN_ROLE grant is missing
```

The intended post-deployment workflow is for the deployer key to be discarded or moved to cold storage, with **finalAdmin** (typically a multisig or governance contract) assuming full

operational control. Without **ADMIN_ROLE**, **finalAdmin** cannot execute any admin-gated function, cannot manage role membership (e.g., adding a new bridge bot or revoking a compromised one), and cannot grant itself the missing role. Since **DEFAULT_ADMIN_ROLE (0x00)** is also never granted to any address, and the vault overrides the default admin hierarchy by making **ADMIN_ROLE** self-administering, there is no recovery path once the deployer key is lost. The vault becomes permanently stuck with its initial configuration, and every remediable vulnerability becomes permanently unfixable.

Recommendation

Add an **ADMIN_ROLE** grant to **finalAdmin** in **DeployWhitelistedVault::_finalizeSetupAndTransferRoles**, alongside the existing operational role grants. The deployer should then renounce its own **ADMIN_ROLE** to complete the handoff cleanly.

```

    deployed.dnCoreWriter.grantRole(deployed.dnCoreWriter.BRIDGE_ROLE(),
    finalAdmin);
    deployed.dnCoreWriter.grantRole(deployed.dnCoreWriter.WITHDRAWAL_ROLE(),
    finalAdmin);

    deployed.dnCoreWriter.grantRole(deployed.dnCoreWriter.TRADING_AGENT_ROLE(),
    finalAdmin);
    + deployed.dnCoreWriter.grantRole(deployed.dnCoreWriter.ADMIN_ROLE(),
    finalAdmin);
    + deployed.dnCoreWriter.renounceRole(deployed.dnCoreWriter.ADMIN_ROLE(),
    broadcaster);

```

3S-M01

Duplicated **depositReceiver** state between **ExchangeRateUpdater** and **Depositor** can cause incorrect idle balance in NAV calculation

Id	3S--M01
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Addressed in #e401da7 .

Description

ExchangeRateUpdater::getIdleTokenBalances aggregates token balances held across the protocol's key addresses, including the **depositReceiver**, to compute total assets for the vault's exchange rate. The resulting sum feeds into **ExchangeRateUpdater::_calculateTotalAssets** and ultimately into **ExchangeRateUpdater::updateExchangeRate**, which sets the vault's NAV.

Both **ExchangeRateUpdater** and **Depositor** maintain their own independent **depositReceiver** state variable. Each contract sets the value in its constructor and exposes a separate admin-only setter (**ExchangeRateUpdater::setDepositReceiver** and **Depositor::setDepositReceiver**). The two variables are expected to hold the same address, since **Depositor** sends deposited funds to its **depositReceiver** at line 87, and **ExchangeRateUpdater** queries the balance of its own **depositReceiver** at line 177 to account for those funds in the NAV.

If an admin updates **depositReceiver** on only one of the two contracts, the values diverge. When **Depositor.depositReceiver** is updated but **ExchangeRateUpdater.depositReceiver** is not, **ExchangeRateUpdater::getIdleTokenBalances** queries the old address where no new deposits land, underreporting idle balances. Conversely, if only **ExchangeRateUpdater.depositReceiver** is updated, it queries an address that does not hold the actual deposited funds, again producing an incorrect figure. In either case the exchange rate derived from **_calculateTotalAssets** will be wrong, mispricing deposits and withdrawals for all vault users.

Recommendation

ExchangeRateUpdater should not maintain its own copy of **depositReceiver**. Instead, it should store a reference to the **Depositor** contract and read **Depositor.depositReceiver** directly whenever **getIdleTokenBalances** is called. This eliminates the duplicated state and guarantees consistency without requiring coordinated admin calls.

```
- address public depositReceiver;
+ address public depositor;
```

```
- balances[i] += IERC20Metadata(assetConfigs[i].token).balanceOf(depositReceiver);
+ balances[i] += IERC20Metadata(assetConfigs[i].token).balanceOf(
+   Depositor(depositor).depositReceiver()
+ );
```

The **ExchangeRateUpdater::setDepositReceiver** setter should be replaced with a **setDepositor** setter, and the constructor should accept a **Depositor** address rather than a **depositReceiver** address.

3S-L01

Admin can bypass whitelist state synchronization by calling **WhitelistedVaultToken::toggleTransferRestrictions** directly

Id	3S--L01
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Addressed in #24b5d38 .

Description

WhitelistedDepositor::toggleWhitelist is the intended entry point for toggling the whitelist state. When called, it updates the **isWhitelistEnabled** flag in the **WhitelistedDepositor** contract and then calls **WhitelistedVaultToken::toggleTransferRestrictions** to synchronize the transfer restriction state in the vault token.

However, because the **finalAdmin** is granted the admin role, and the admin role has permission to call **WhitelistedVaultToken::toggleTransferRestrictions** directly, the **finalAdmin** can invoke this function without going through **WhitelistedDepositor::toggleWhitelist**. This desynchronizes the two contracts: the vault token's **transfersRestricted** is set to **false** while **isWhitelistEnabled** in **WhitelistedDepositor** remains **true**. The protocol cannot recover from this state because **WhitelistedVaultToken::toggleTransferRestrictions** contains a guard **if (!transfersRestricted)** that prevents the function from executing when **transfersRestricted** is already **false**. As a result, any subsequent call to **WhitelistedDepositor::toggleWhitelist** attempting to re-synchronize the state will be unable to call through to the vault. The vault token becomes freely transferable while the whitelist cap is permanently enforced at the depositor level.

Steps to Reproduce

1. **finalAdmin** calls **WhitelistedVaultToken::toggleTransferRestrictions** directly instead of going through **WhitelistedDepositor::toggleWhitelist**.
2. **transfersRestricted** in **WhitelistedVaultToken** is set to **false** (transfer restrictions lifted).
3. **isWhitelistEnabled** in **WhitelistedDepositor** remains **true** (whitelist cap still enforced at the depositor level).
4. **finalAdmin** attempts to call **WhitelistedDepositor::toggleWhitelist** to correct the state.

5. The depositor attempts to call **WhitelistedVaultToken::toggleTransferRestrictions**, but the vault's **if (!transfersRestricted)** guard blocks the call since **transfersRestricted** is already **false**.

6. The state is permanently stuck: the vault token is freely transferable, but the whitelist deposit cap remains active in the depositor.

Recommendation

Option 1 - Contract level fix: Restrict **WhitelistedVaultToken::toggleTransferRestrictions** so that only the **WhitelistedDepositor** contract can call it, and remove the admin role from the depositor in the deployment script. This eliminates the possibility of the admin bypassing the intended flow entirely.

In **WhitelistedVaultToken**, replace the **requiresAuth** modifier with an explicit caller check:

```
- function toggleTransferRestrictions() external requiresAuth {
+ function toggleTransferRestrictions() external {
+   require(msg.sender == address(whitelistedDepositor), "only depositor");
+   transfersRestricted = !transfersRestricted;
+ }
```

In the deployment script, remove the step that grants the admin role permission to call **toggleTransferRestrictions** on the vault token, and remove the admin role from the **WhitelistedDepositor** contract:

```
- // Grant admin role to WhitelistedDepositor
- vault.setRoleCapability(ADMIN_ROLE, address(whitelistedDepositor),
WhitelistedDepositor.toggleWhitelist.selector, true);
- // Grant admin role permission to call toggleTransferRestrictions
- vault.setRoleCapability(ADMIN_ROLE, address(whitelistedVaultToken),
WhitelistedVaultToken.toggleTransferRestrictions.selector, true);
```

Option 2 - Documentation: Clearly document that the admin must always use **WhitelistedDepositor::toggleWhitelist** to toggle the whitelist state and must never call **WhitelistedVaultToken::toggleTransferRestrictions** directly. The documentation should explain that bypassing the depositor will permanently desynchronize the two contracts with no recovery path, since the vault's **if (!transfersRestricted)** guard will block all future synchronization attempts.

3S-L02

Default perp DEX index can be registered in **perpDexIndices**, causing double-counting of account value

Id	3S--L02
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #37c81fc .

Description

ExchangeRateUpdater::addPerpDexIndex allows authorized callers to register additional HIP-3 builder DEX indices whose perp account values are tracked as part of the vault's total assets. In **ExchangeRateUpdater::readAccountMarginSummaryUSDC**, the default perp DEX (index **0**) is always queried unconditionally via a hardcoded call to **I1Read.accountMarginSummary(0, dnCoreWriters[i])**, and then all entries in **perpDexIndices** are iterated and queried as well. Because **addPerpDexIndex** only guards against duplicate entries already present in **perpDexIndices**, it does not prevent index **0** from being added. If a caller registers **_dexIndex == 0**, the default DEX account value will be fetched twice per writer address, inflating the result of **readAccountMarginSummaryUSDC** and, consequently, the vault's reported total assets and exchange rate.

Recommendation

Add an explicit check at the top of **addPerpDexIndex** to reject **_dexIndex == 0**, since the default DEX is always accounted for separately and must never appear in **perpDexIndices**.

```
function addPerpDexIndex(uint32 _dexIndex) external requiresAuth {
+   require(_dexIndex != 0, "DEX index 0 is reserved for default perps");
   for (uint256 i = 0; i < perpDexIndices.length; i++) {
       require(perpDexIndices[i] != _dexIndex, "DEX index already exists");
   }
   perpDexIndices.push(_dexIndex);
}
```


}

3S-L03

WhitelistedDepositor::remainingDepositCapFor returns an incorrect cap when the vault cap is the binding constraint

Id	3S--L03
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #855fb44 .

Description

WhitelistedDepositor::remainingDepositCapFor is intended to return the maximum pre-fee token amount a user can pass to **deposit()** given both their personal whitelist cap and the global vault cap. It is the primary view function that integrators and users rely on to determine how much they can deposit without reverting.

The two helper functions it aggregates are not expressed in the same unit:

- **remainingUserDepositCap** back-calculates and returns a **pre-fee** amount (i.e. the raw **_amount** that can be passed directly to **deposit()**).
- **remainingVaultDepositCap** derives the remaining capacity from **pricer.getAssetAmount** applied to the remaining share supply, which reflects **post-fee** asset value (since the fee is deducted before shares are minted).

When **vaultCap < userCap**, the function returns **vaultCap** as-is. Because **vaultCap** is a post-fee amount, any caller who uses this return value as their **deposit()** **_amount** will deposit less than the vault actually allows, since the fee portion of their deposit would consume no vault capacity.

Recommendation

Before comparing the two caps, convert **vaultCap** from post-fee to pre-fee terms using the same inversion applied in **remainingUserDepositCap**:

```
function remainingDepositCapFor(address _user) external view returns (uint256) {
    uint256 userCap = remainingUserDepositCap(_user);
    uint256 vaultCap = remainingVaultDepositCap();
```

```
+    if (depositFee > 0 && depositFeeRecipient != address(0)) {  
+        vaultCap = vaultCap.mulDiv(BASE, BASE - depositFee);  
+    }  
    return userCap < vaultCap ? userCap : vaultCap;  
}
```

This ensures both values are expressed in the same pre-fee units and the return value can be used directly as the **_amount** argument to **deposit()**.

3S-L04

Missing zero-address validation in **setWithdrawalQueue** and **setCoreDepositWallet** allows silent misconfiguration that bricks withdrawals and bridging

Id	3S--L04
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #8b0a6f4 .

Description

DnCoreWriterVault::setWithdrawalQueue and **DnCoreWriterVault::setCoreDepositWallet** are admin-restricted setters that configure two operationally critical addresses: the withdrawal queue that receives tokens during **releaseTokensForWithdrawals**, and the core deposit wallet used to bridge USDC into HyperCore via **bridgeUSDCToHypercore**.

Neither function validates that the supplied address is non-zero. An admin that accidentally passes **address(0)** to **setWithdrawalQueue** will silently overwrite a previously valid withdrawal queue address. Subsequent calls to **DnCoreWriterVault::releaseTokensForWithdrawals** will revert on the existing **address(0)** guard, effectively halting all user withdrawals until the admin notices and corrects the configuration.

If **coreDepositWallet** is set to **address(0)**, **DnCoreWriterVault::bridgeUSDCToHypercore** will attempt **ICoreDepositWallet(address(0)).deposit(...)**, which will revert.

Recommendation

Add **address(0)** checks to both setters so that misconfiguration reverts immediately at the point of the erroneous call rather than surfacing later in unrelated operations.

```
function setWithdrawalQueue(address _withdrawalQueue) external
onlyRole(ADMIN_ROLE) {
```

```
+   require(_withdrawalQueue != address(0), "DnCoreWriter: zero address");
    withdrawalQueue = _withdrawalQueue;
    emit WithdrawalQueueSet(_withdrawalQueue);
}
function setCoreDepositWallet(address _coreDepositWallet) external
onlyRole(ADMIN_ROLE) {
+   require(_coreDepositWallet != address(0), "DnCoreWriter: zero address");
    coreDepositWallet = _coreDepositWallet;
}
```

3S-N01

WhitelistedDepositor::remainingUserDepositCap reverts due to arithmetic underflow when cap is reduced below cumulative deposited amount

Id	3S--N01
Classification	None
Category	Suggestion
Status	Addressed in #b1d1d6a .

Description

WhitelistedDepositor::remainingUserDepositCap is a view function that returns how much a user can still deposit under their personal whitelist cap, after accounting for any deposit fee. It is also called internally by **remainingDepositCapFor**, which returns the effective remaining capacity as the minimum of the user cap and the global vault cap.

The function computes **cap - cumulativeDeposited[_user]**. If **cumulativeDeposited[_user]** exceeds **cap**, this subtraction underflows and reverts. Under normal deposit flow this cannot happen, but it becomes reachable when an admin calls **setWhitelistCap** or **setWhitelistCapBatch** to lower a user's **whitelistCap** to a value below what the user has already deposited. There is no validation in either setter to prevent this.

Steps to Reproduce

1. Admin whitelists a user with a cap of 1000 USDC via **setWhitelistCap(user, 1000e6)**.
2. The user deposits 800 USDC. **cumulativeDeposited[user]** is now **800e6**.
3. Admin decides to reduce the user's cap and calls **setWhitelistCap(user, 500e6)**. The call succeeds with no revert.
4. Any call to **remainingUserDepositCap(user)** or **remainingDepositCapFor(user)** now reverts with a panic due to arithmetic underflow (**500e6 - 800e6**).

Recommendation

Add a saturation check before the subtraction in **WhitelistedDepositor::remainingUserDepositCap** so that a cap set below the already-deposited amount returns **0** instead of reverting. This makes the view function unconditionally safe regardless of how admins configure caps.

```

function remainingUserDepositCap(address _user) public view returns (uint256) {
    if (!isWhitelistEnabled) return type(uint256).max;
    uint256 cap = whitelistCap[_user];
-   if (cap == 0) return 0;
+   if (cap == 0 || cap <= cumulativeDeposited[_user]) return 0;
    uint256 postFeeRemaining = cap - cumulativeDeposited[_user];
    if (depositFee == 0 || depositFeeRecipient == address(0)) return
postFeeRemaining;
    return postFeeRemaining.mulDiv(BASE, BASE - depositFee);
}

```

3S-N02

Unbounded **usdc** parameter in **bridgeUSDCToHypercore** enables arbitrary token approval

Id	3S--N02
Classification	None
Category	Suggestion
Status	Addressed in #ad02bf3 .

Description

DnCoreWriterVault::bridgeUSDCToHypercore exists as a special-case bridging path for USDC, required because USDC's HyperCore system address is blacklisted and the general-purpose **bridgeERC20ToHypercore** path reverts for it. The function accepts an **address usdc** parameter and grants a **safeIncreaseAllowance** on that address to the **coreDepositWallet**.

```
function bridgeUSDCToHypercore(address usdc, uint256 amount, uint32
destinationDex) external onlyRole(BRIDGE_ROLE) {
    IERC20Metadata(usdc).safeIncreaseAllowance(coreDepositWallet, amount);
    ICoreDepositWallet(coreDepositWallet).deposit(amount, destinationDex);
    emit BridgeERC20Hypercore(usdc, coreDepositWallet, amount);
}
```

Nothing constrains **usdc** to the actual native USDC token address. A caller with **BRIDGE_ROLE** can pass any ERC-20 address, causing the vault to grant an allowance on an unrelated token to **coreDepositWallet**. The practical impact is limited because **BRIDGE_ROLE** is already a trusted privileged role whose compromise would have broader consequences, but the lack of validation violates the principle of least authority and widens the blast radius of a compromised bridge key.

Recommendation

Validate that the **usdc** parameter matches a known USDC address. This can be done by storing the USDC address as an immutable and requiring it at the call site.

```
+ address public immutable USDC;
function bridgeUSDCToHypercore(address usdc, uint256 amount, uint32
```



```
destinationDex) external onlyRole(BRIDGE_ROLE) {  
+   require(usdc == USDC, "Invalid USDC address");  
   IERC20Metadata(usdc).safeIncreaseAllowance(coreDepositWallet, amount);  
   ICoreDepositWallet(coreDepositWallet).deposit(amount, destinationDex);  
   emit BridgeERC20Hypercore(usdc, coreDepositWallet, amount);  
}
```

Alternatively, remove the **usdc** parameter entirely and read the USDC address from storage, eliminating the caller-controlled input.

3S-N03

Missing event emissions in admin functions hinders off-chain monitoring

Id	3S--N03
Classification	None
Category	Suggestion
Status	Addressed in #80bff36 .

Description

Several admin-gated functions that mutate protocol configuration state do not emit events, breaking consistency with peer functions in the same contracts and reducing off-chain observability.

In **ExchangeRateUpdater**, both **addPerpDexIndex** and **removePerpDexIndex** silently modify the **perpDexIndices** array without emitting any event. Every other add/remove pair in the contract (**addDnCoreWriterAddress/removeDnCoreWriterAddress**, **addSpotPositionConfig/removeSpotPositionConfig**, **addAssetConfig/removeAssetConfig**) emits a corresponding event on success.

In **DnCoreWriterVault**, **setCoreDepositWallet** updates the **coreDepositWallet** storage variable without emitting an event, while the analogous **setWithdrawalQueue** emits **WithdrawalQueueSet**. Similarly, **registerToken** writes a full **HyperCoreTokenConfig** struct into **tokenConfig** without any event, while other config-toggling functions like **toggleWithdrawalToken** and **toggleTradeableAsset** emit events.

Without these events, off-chain monitoring systems, indexers, and governance dashboards cannot detect when perp DEX indices are added or removed, when the core deposit wallet is changed, or when new tokens are registered. This makes it harder to audit admin activity and react to potentially malicious configuration changes.

Recommendation

Define and emit events for each of the affected functions, following the naming conventions already established in the respective contracts.

For **ExchangeRateUpdater**:

```
+ event PerpDexIndexAdded(uint32 dexIndex);
```

```
+ event PerpDexIndexRemoved(uint32 dexIndex);
```

```
function addPerpDexIndex(uint32 _dexIndex) external requiresAuth {
    for (uint256 i = 0; i < perpDexIndices.length; i++) {
        require(perpDexIndices[i] != _dexIndex, "DEX index already exists");
    }
    perpDexIndices.push(_dexIndex);
+   emit PerpDexIndexAdded(_dexIndex);
}
```

```
function removePerpDexIndex(uint32 _dexIndex) external requiresAuth {
    for (uint256 i = 0; i < perpDexIndices.length; i++) {
        if (perpDexIndices[i] == _dexIndex) {
            perpDexIndices[i] = perpDexIndices[perpDexIndices.length - 1];
            perpDexIndices.pop();
+           emit PerpDexIndexRemoved(_dexIndex);
            return;
        }
    }
}
```

For DnCoreWriterVault:

```
+ event CoreDepositWalletSet(address indexed coreDepositWallet);
+ event TokenRegistered(address indexed token, address indexed systemAddress,
uint64 tokenId);
```

```
function setCoreDepositWallet(address _coreDepositWallet) external
onlyRole(ADMIN_ROLE) {
    coreDepositWallet = _coreDepositWallet;
+   emit CoreDepositWalletSet(_coreDepositWallet);
}
```

```
function registerToken(
    address token,
    address systemAddress,
```

```

uint64 tokenId,
uint8 hyperEVMDecimals,
uint8 hypercoreDecimals
)
external
onlyRole(ADMIN_ROLE)
{
    tokenConfig[token] = HyperCoreTokenConfig({
        tokenId: tokenId,
        systemAddress: systemAddress,
        hyperEVMDecimals: hyperEVMDecimals,
        hypercoreDecimals: hypercoreDecimals
    });
+   emit TokenRegistered(token, systemAddress, tokenId);
}

```